Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Подсистема обнаружения сигналов с цифровой модуляцией, реализованная на ПЛИС

Фаст Никита Михайлович

Отчёт по учебной практике
в форме «Производственное задание»

Научный руководитель:
Профессор кафедры системного программирования, д. т. н., Д. В. Кознов

Консультант:
Начальник отдела программирования АО «Концерн ГРАНИТ», А. С. Кривоногов

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **6**

2. Обзор **7**

2.1. Особенности процесса разработки под ПЛИС 7

2.2. Используемые технологии 8

2.3. Существующие решения 10

2.4. Корреляционное обнаружение сигналов 11

3. Архитектура **14**

3.1. Место ПЛИС-обнаружителя в системе 15

3.2. Формат входных данных 15

3.3. Назначение и интерфейсы обнаружителя 16

4. Архитектура ПЛИС-обнаружителя **17**

4.1. Общая организация вычислительного тракта 17

4.2. Корреляция через БПФ 18

4.3. Переход от циклической свёртки к линейной 19

4.4. Формирование и хранение спектров преамбул 20

4.5. Обработка спектра входного сигнала 20

4.6. Обратное БПФ и формирование выходного потока 21

4.7. Организация управления потоками 21

5. Тестирование **23**

5.1. Параметры тестовой симуляции 23

5.2. Организация моделирования 24

5.3. Результаты 25

Заключение **26**

Список литературы **28**

Введение

Современные системы связи и радиолокации распространены в разных областях деятельности человека. Речь идет о таких системах как сети высокоскоростного мобильного интернета 4G LTE и 5G или радиостанциях класса DMR, используемых для обеспечения сухопутной мобильной радиосвязи. Неотъемлемой частью таких систем является функция обнаружение сигналов, где под обнаружением понимается определение момента времени начала сигнала и его смещения по частоте относительно центральной частоты приема.

Данная задача является сложной в силу наличия в радиоэфире шумов и помех, неидеальностью приёмной и передающей аппаратуры, а также эффектов многолучевого распространения сигналов (доплеровский сдвиг частоты, искажение формы сигнала).

Оптимальным методом обнаружения сигналов на фоне аддитивного белого Гауссовского шума (АБГШ) является корреляционное обнаружения сигналов [25]. Основная идея этого метода заключается в том, что в сигнал на передающей стороне помещается специальная последовательность (преамбула), которая заранее известна как передатчику, так и приемнику. При приеме сигнала осуществляется вычисление корреляции между принятым сигналом и сохраненным в памяти эталонным образцом. В силу того, что корреляция двух сигналов является мерой их сходства, при появлении в принимаемом сигнале преамбулы значение корреляции резко возрастает, что позволяет идентифицировать начало полезного сигнала.

Для вычисления корреляционного интеграла при частоте дискретизации сигнала fs и длине преамбулы в L отсчетов требуется выполнить fs * L операций умножения-сложения (multiply-accumulate, MAC) за секунду. Соответственно, для системы цифровой связи с частотой дискретизации 400 kHz и длиной преамбулы в 1024 отсчета для расчета корреляционного интеграла потребуется вычислитель с производительностью не менее 410 MIPS, что, в принципе, не является проблемой для современного аппаратного обеспечения.

Однако принимаемый сигнал содержит различные искажения, в числе которых особенно важна ошибка по частоте [13]. Наличие частотной ошибки в сигнале препятствует расчету корреляции, поскольку преамбула с частотной ошибкой не совпадает с эталонной и, следовательно, полученное значение корреляции будет ниже порогового. Таким образом, наличие частотной ошибки приводит к деградации корреляционного пика и делает обнаружение сигнала невозможным. В связи с этим эффект частотной ошибки должен быть нивелирован, что достигается путем определения величины частотной ошибки и выполнения частотной коррекции сигнала на найденную величину.

Необходимость компенсации частотной ошибки приводит к значительному увеличению вычислительной сложности метода корреляционного обнаружения сигналов. При переборе N различных частотных ошибок необходимо выполнить N раз расчет корреляционного интеграла, что приводит к увеличению объема вычислений в N раз (типовым значением N является 100). Таким образом, реализация метода корреляционного обнаружения сигналов для системы связи, рассмотренной ранее, потребует производительность вычислителя уже в 41000 MIPS, что является вызовом для современных вычислителей.

В качестве основного вычислителя для систем радиолокации и связи могут выступать цифровые сигнальные процессоры, интегральные схемы и ПЛИС [5]. Основными производителями сигнальных процессоров являются компании Texas Instruments и Analog Devices (США), которые выпускаю семейства процессоров TMS320C64xx [19] и TigerSHARC [10] соответственно. Использование сигнальных процессоров оправдано при производстве мелкосерийных или средне серийных изделий, требующих для своей работы умеренную производительность. В случае крупносерийного производства экономически оправданным становится переход к использованию интегральных схем. В данном сценарии также появляется интерес к ПЛИС (например AMD 7 series FPGA [22] или Altera Cyclone IV FPGA [6]) как к средству прототипирования целевой интегральной схемы. Также выбор ПЛИС может быть обусловлен высокой вычислительной сложностью решаемой задачи, для которой произво-

дительности сигнального процессора будет недостаточно [8].

Одной из платформ, сочетающей интерфейс с радиоканалом (микросхема, обеспечивающая прием и передачу радиосигналов), ПЛИС и процессор, является технология программно-определяемой радиосистемы (Software Defined Radio, SDR) [18]. В рамках данной технологии реализация ряда компонентов системы связи выполняется средствами программными, а не аналоговыми, что позволяет осуществлять перенастройку широкого спектра параметров прямо во время работы SDR. Данным обстоятельством, в совокупности со способностью современных ПЛИС к выполнению частичной переконфигурации без нарушения работоспособности остальной части ПЛИС, объясняется высокая гибкость SDR-решений и интерес к использованию программно-определяемых радиосистем в качестве платформы для разработки и прототипирования систем цифровой связи.

В рамках технологии SDR реализация ресурсоемкого метода корреляционного обнаружения сигналов непосредственно на ПЛИС позволяет разгрузить центральный процессор системы для выполнения других задач, таких как демодуляция, декодирование, протокольная обработка и управление системой в целом.

АО “Концерн ГРАНИТ” занимается разработкой современных систем связи и радиоэлектроники для профильных министерств и ведомств Российской Федерации. В рамках работ по созданию беспроводных систем связи было необходимо разработать обнаружитель цифровых сигналов, работающий на ПЛИС. В качестве платформы для реализации обнаружителя было выдвинуто требование по использованию устройства Pluto SDR+, которое является клоном ADALM-PLUTO SDR [1] производства Analog Devices, и представляет собой доступную реализацию SDR на базе системы на кристалле (System on a Chip, SoC) Xilinx Zynq-7000 [24], сочетающую программируемую логику и двухъядерный процессор ARM.

1. Постановка задачи

Целью работы является разработка подсистемы для устройства Pluto SDR+, осуществляющей эффективную реализацию на уровне регистровых передач алгоритма обнаружения сигналов с цифровой модуляцией на ПЛИС с использованием языка программирования VHDL и апробацией на устройстве. Для достижения данной цели были поставлены следующие задачи.

1. Выполнить обзор существующих и доступных алгоритмов обнаружения цифровых сигналов на ПЛИС, а также проблематики их реализации на ПЛИС. Выбрать подходящий алгоритм.

2. Спроектировать архитектуру ПЛИС-обнаружителя с учетом дальнейшей интеграцию в устройство Pluto SDR+.

3. Реализовать обнаружитель на языке VHDL.

4. Выполнить тестирование и апробацию полученного решения.

2. Обзор

2.1. Особенности процесса разработки под ПЛИС

Программируемые логические интегральные схемы (ПЛИС) – это класс цифровых интегральных схем, логика которых не определяется на этапе производства, а конфигурируется пользователем после изготовления. ПЛИС состоит из набора логических блоков и множества программируемых соединений. Логические блоки содержат примитивы, такие как: D-триггеры, таблицы перекодировки (Lookup Table, LUT), DSP-блоки, мультиплексоры, порты ввода-вывода и блоки памяти.

Важно отметить, что процесс разработки для ПЛИС существенно отличается от процесса программирования процессора, поскольку ПЛИС, в отличие от процессора, не выполняет последовательность инструкций. В случае ПЛИС код, написанный на языке описания аппаратуры (Hardware Description Language, HDL), описывает структуру и поведение цифровой схемы. На основе данного описания инструменты автоматизированного проектирования формируют конфигурацию внутренних ресурсов микросхемы, обеспечивающую реализацию заданного алгоритма.

Для разработки под ПЛИС используются программные средства, предоставляемые производителем микросхемы. Современные средства разработки включают интегрированную среду проектирования (IDE), обеспечивающую полный цикл проектирования. Основной задачей такой среды является преобразование HDL-кода в конфигурацию логических примитивов, имеющихся в ПЛИС, их размещение и маршрутизация соединений. В связи с этим основными компонентами IDE являются: редактор HDL-кода, синтезатор, средство размещения и трассировки (place-and-route), а также генератор конфигурационного файла (битового потока). Разработчик может оказывать влияние на синтез и размещение при помощи ограничений (constraints), задающих временные, топологические и электрические параметры проекта.

Результатом разработки на языках HDL является описание аппарат-

ной структуры, которое может быть оформлено в виде повторно используемых модулей, называемых IP-ядрами. IP-ядро может использоваться как самостоятельно, так и входить в состав более сложных систем. При необходимости RTL-описание проекта может быть использовано в дальнейшем для разработки интегральной схемы специального назначения (Application-Specific Integrated Circuit, ASIC), предназначенной для серийного производства.

В настоящее время ПЛИС зачастую комбинируются с процессорным ядром на одном кристалле, образуя систему на кристалле (System-on-Chip, SoC). Такая система состоит из двух основных подсистем: процессорной системы (Processing System, PS) и программируемой логики (Programmable Logic, PL). Взаимодействие между подсистемами осуществляется по стандартизированным интерфейсам (например, AXI). Наличие процессорной подсистемы позволяет выполнять высокоуровневое управление, конфигурацию и отладку аппаратных модулей. Производительности процессорной системы, как правило, достаточно для развертывания операционной системы Linux[12, 23]. При отсутствии необходимости использования полноценной операционной системы возможно создание приложений типа Bare-Metal. Интеграция процессорной системы и программируемой логики расширяет функциональные возможности устройства и упрощает процесс верификации и отладки разрабатываемых аппаратных модулей. В связи с этим одним из распространённых сценариев применения SoC является организация платформ для разработки, отладки и апробации IP-ядер.

2.2. Используемые технологии

2.2.1. Платформа

Задачей разрабатываемого обнаружителя является обнаружение радиосигналов в заданном диапазоне частот и оценка их параметров. Для функционирования обнаружителя требуется входной цифровой сигнал, сформированный на основе принимаемого радиочастотного сигнала. Следовательно, аппаратная платформа должна обеспечивать не только вы-

числительные ресурсы ПЛИС, но и средства радиочастотного приёма и аналого-цифрового преобразования.

Таким образом, для реализации системы обнаружения требуется устройство, включающее систему на кристалле (SoC) и радиочастотный приёмопередающий тракт. Подобные устройства относятся к классу программно определяемых радиосистем (Software Defined Radio, SDR), в которых значительная часть обработки сигнала выполняется программно или аппаратно-программными средствами.

В качестве аппаратной платформы была выбрана модифицированная версия устройства ADALM-PLUTO (Pluto SDR+), в которой посредством изменения конфигурации расширен рабочий диапазон частот с 325 МГц–3,8 ГГц до 70 МГц–6 ГГц. Данное устройство сочетает в себе вычислительные ресурсы и радиочастотный тракт и обладает оптимальным соотношением стоимости и функциональных возможностей для решения поставленной задачи.

Основными компонентами выбранной платформы являются:

- система на кристалле AMD Zynq-7000 SoC, включающая процессорную подсистему и программируемую логическую часть;
- микросхема приемопередатчика AD9363[16], представляющая собой интегрированный RF-трансивер со встроенными аналого-цифровыми и цифро-аналоговыми преобразователями и цифровым интерфейсом передачи I/Q-отсчётов.

2.2.2. Используемые инструменты

При разработке проектов для ПЛИС выбор инструментов во многом определяется производителем целевой микросхемы. Как отмечалось ранее, код, написанный на языках HDL, не исполняется непосредственно устройством, а представляет собой описание структуры и поведения разрабатываемой цифровой схемы. На основе данного описания формируется конфигурация аппаратных ресурсов ПЛИС.

Преобразование RTL-описания в технологически зависимое представление, состоящее из примитивов целевой архитектуры, выполня-

ется инструментом синтеза. Процесс синтеза требует информации о составе и характеристиках ресурсов выбранной ПЛИС. После этапа синтеза выполняются процедуры размещения (placement) и трассировки (routing), а также статический временной анализ (Static Timing Analysis), позволяющий проверить корректность работы схемы с учётом заданных временных ограничений. Все перечисленные этапы требуют детального знания архитектуры устройства, поэтому официальные средства разработки предоставляются производителем ПЛИС.

В рассматриваемой работе используется система на кристалле семейства Zynq-7000, разработанная компанией AMD (ранее Xilinx). В качестве основной среды аппаратного проектирования применяется пакет Vivado Design Suite[3], предназначенный для синтеза, реализации и генерации конфигурационного файла.

Поскольку используемая платформа представляет собой SoC с процессорной подсистемой на базе ARM, для разработки программного обеспечения на языках C и C++ и его последующей загрузки и отладки используется среда AMD Vitis Unified Software Platform[2].

В качестве языка описания аппаратуры (Hardware Description Language, HDL) могут использоваться VHDL[20], Verilog[21] и SystemVerilog[11], которые получили широкое распространение в промышленной практике. Среда Vivado поддерживает данные языки. Для реализации обнаружителя в рамках данной работы был выбран язык VHDL, поскольку в организации АО концерн «ГРАНИТ» данный язык является основным.

2.3. Существующие решения

Разработанные IP-ядра зачастую обладают высокой коммерческой ценностью, поскольку на их основе могут быть созданы серийно производимые интегральные схемы. В силу этого значительная часть промышленных IP-ядер распространяется на проприетарной основе и недоступна для свободного использования и модификации.

В то же время существуют проекты и сообщества, разрабатывающие IP-ядра с открытым исходным кодом. К числу таких ресурсов относят-

ся специализированные каталоги и репозитории, например OpenCores[14], FuseSoC[9], а также агрегированные коллекции VHDL/Verilog IP-ядер[7]. Подобные решения, как правило, ориентированы на реализацию базовых интерфейсов, периферийных контроллеров или типовых функциональных блоков и могут иметь ограничения по уровню оптимизации, поддержке и адаптации под конкретные аппаратные платформы.

Дополнительно необходимо учитывать, что IP-ядро может быть реализовано с использованием вендор-специфичных примитивов и библиотек, что приводит к его несовместимости с ПЛИС другого производителя. Хотя RTL-описание в общем случае является переносимым, практическая реализация часто зависит от доступных аппаратных ресурсов, таких как блоки памяти, DSP-модули или специализированные интерфейсы. Возможен также сценарий, при котором IP-ядро не может быть реализовано на младших моделях ПЛИС того же семейства из-за недостаточного объёма доступных ресурсов.

Каждый из перечисленных факторов может служить обоснованием необходимости разработки собственного IP-ядра. В данной работе рассматривается создание собственного IP-ядра обнаружителя для платформы AMD Zynq-7000 SoC[4], поскольку проведённый обзор доступных открытых IP-ядер показал отсутствие готовых реализаций обнаружителя.

2.4. Корреляционное обнаружение сигналов

При обнаружении сигналов широко применяется подход, основанный на использовании преамбулы — заранее известной последовательности отсчётов, обладающей хорошими автокорреляционными свойствами.

Приёмник выполняет сравнение принятого сигнала с опорной преамбулой посредством вычисления взаимнокорреляционной функции, которая количественно характеризует степень сходства двух сигналов [**?**].

Взаимнокорреляционная функция для дискретных сигналов может

быть записана в виде

$$R_{xy}(k) = \sum_{n=0}^{N-1} x(n)\, y^*(n-k), \tag{1}$$

где $x(n)$ — принятый сигнал, $y(n)$ — опорная преамбула, $(\cdot)^*$ обозначает комплексное сопряжение, N — длина преамбулы.

При наличии сигнала максимум взаимнокорреляционной функции существенно превышает уровень отклика, формируемого шумом, что позволяет судить о наличии сигнала и его временном положении. Амплитуда корреляционного пика пропорциональна энергии сигнала.

Введём две гипотезы:

$$H_0 : x(n) = w(n),$$

$$H_1 : x(n) = s(n) + w(n),$$

где $w(n)$ — аддитивный белый гауссовский шум, $s(n)$ — искомый сигнал.

Решение об обнаружении принимается по правилу

$$\max_k |R_{xy}(k)| \underset{H_0}{\overset{H_1}{\gtrless}} \gamma, \tag{2}$$

где γ — порог обнаружения, выбираемый исходя из допустимой вероятности ложного срабатывания.

В отсутствии сигнала отклик корреляционного обнаружителя представляет собой случайную величину с известным распределением, что позволяет задать порог γ в соответствии с требуемой вероятностью ложной тревоги.

Корреляционный обнаружитель является оптимальным для канала с аддитивным белым гауссовским шумом при известной форме сигнала [**?**]. По этой причине в работе рассматривается разработка корреляционного обнаружителя.

Принятый радиосигнал отличается от переданного вследствие воздействия шумов приёмного тракта, внешних помех, эффектов многолучевого распространения, а также нестабильности опорных генераторов

передающей и приёмной сторон.

Поскольку корреляционный обнаружитель основан на когерентном суммировании отсчётов, наличие частотного рассогласования Δf между сигналом и опорой приводит к фазовому вращению во времени и снижению амплитуды корреляционного пика (рис. 1).

Поиск частотного смещения осуществляется дискретным перебором в пределах заранее известного диапазона. Непрерывный перебор невозможен, поэтому диапазон частот дискретизируется с некоторым шагом Δf_{step}.

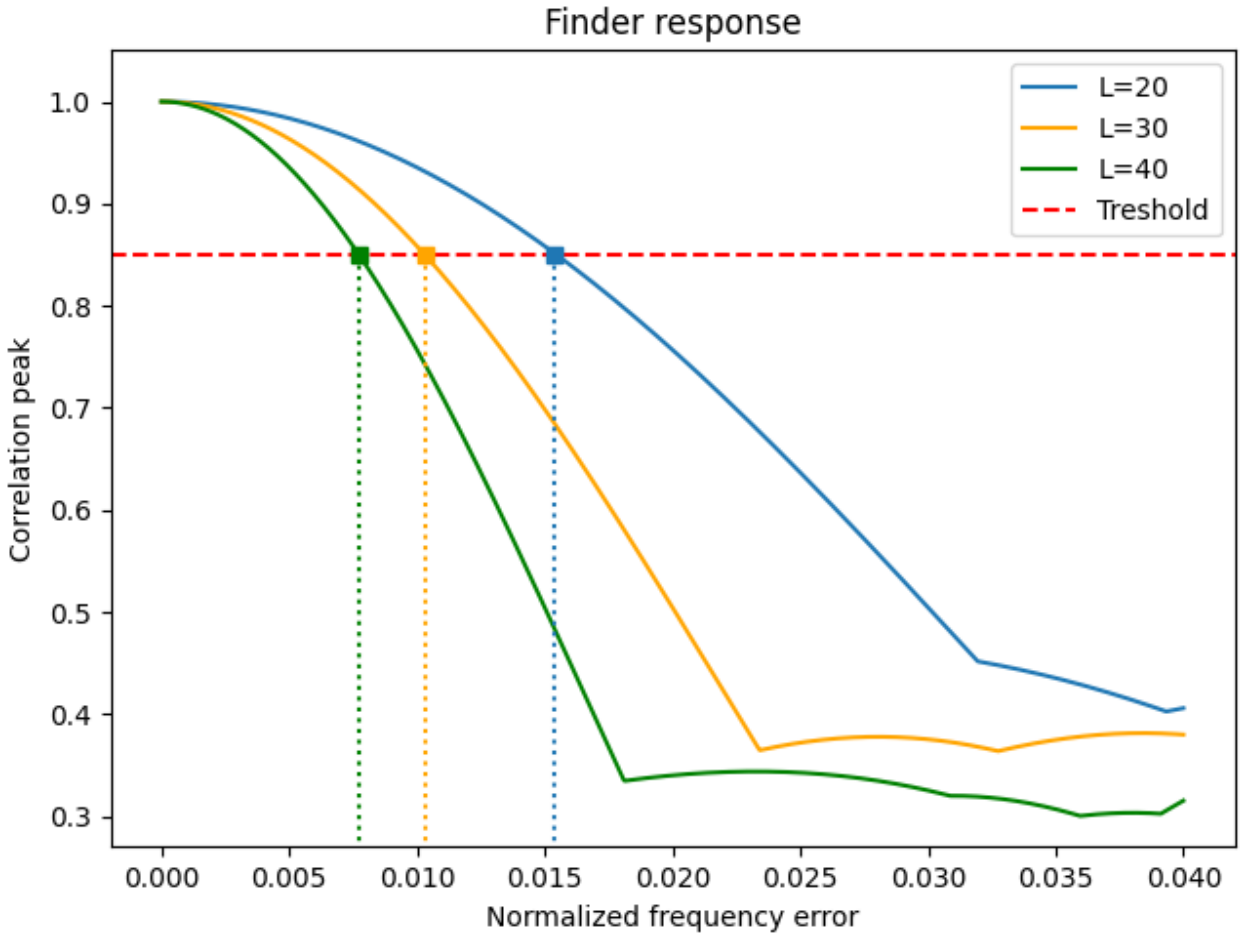
Величина шага связана с длительностью преамбулы. Чем больше длина преамбулы, тем уже главный лепесток функции неопределённости и тем меньше допустимая остаточная частотная ошибка.

Если шаг перебора слишком велик, остаточная частотная ошибка может привести к снижению корреляционного пика ниже порогового уровня и, как следствие, к пропуску сигнала. С другой стороны, уменьшение шага увеличивает число вычислительных операций и требования к вычислительным ресурсам.

Увеличение длины преамбулы приводит к росту её энергии при фиксированной мощности передатчика, что повышает дальность обнаружения сигнала. Однако увеличение длины преамбулы также повышает вычислительную нагрузку и усложняет процедуру частотного поиска.

Таким образом, выбор длины преамбулы представляет собой компромисс между энергетическими характеристиками обнаружителя и вычислительной сложностью его реализации.

Рис. 1: Влияние частотной ошибки на отклик корреляционного обнаружителя в зависимости от длины преамбулы.



3. Архитектура

Разработка ПЛИС-обнаружителя выполняется для платформы PLutoSDR+ построенной на базе SoC (ARM + FPGA) и приемопередатчика AD9363.

В качестве основы используется открытая HDL-прошивка из репозитория[1] компании Analog Devices. Для устройства PLuto SDR+ применяется проект[2], который содержит как аппаратную (HDL), так и программную (C) части системы.

Программный код, исполняемый на процессоре ARM, выполняет:

- инициализацию периферии;
- конфигурирование AD9363 (по протоколу SPI);

[1] https://github.com/analogdevicesinc/hdl
[2] https://github.com/analogdevicesinc/hdl/tree/main/projects/pluto

- запуск механизма передачи данных через DMA.

После настройки приемника оцифрованные отсчёты АЦП поступают в ПЛИС, где далее передаются по внутреннему цифровому тракту и могут быть считаны процессором через DMA с размещением данных в памяти DDR.

3.1. Место ПЛИС-обнаружителя в системе

Разрабатываемый ПЛИС-обнаружитель встраивается в существующую HDL-архитектуру после узла формирования комплексных отсчётов АЦП и до блока DMA.

Таким образом, поток данных в модифицированной системе имеет вид:

$$\text{AD9363} \rightarrow \text{входной интерфейс ПЛИС} \rightarrow \text{ПЛИС-обнаружитель} \rightarrow \text{DMA} \rightarrow \text{DDR}$$

Обнаружитель функционирует полностью внутри логической части (PL) и не требует промежуточной передачи данных в процессорную систему (PS). Это позволяет выполнять корреляционную обработку непосредственно в потоке входных данных без дополнительной нагрузки на интерфейсы PS–PL.

3.2. Формат входных данных

Приемопередатчик AD9363 формирует поток комплексных отсчётов сигнала. Каждый отсчёт представлен 24-битным числом комплексного формата:

- 12 бит — действительная компонента (I);
- 12 бит — мнимая компонента (Q).

На выходе данного узла формируется синхронный поток комплексных отсчётов I/Q фиксированной разрядности, тактируемый сигналом, производным от тактовой частоты АЦП. Именно этот поток подаётся на вход ПЛИС-обнаружителя.

3.3. Назначение и интерфейсы обнаружителя

Разрабатываемый ПЛИС-обнаружитель предназначен для вычисления взаимнокорреляционной функции входного комплексного сигнала с эталонной преамбулой при заданном наборе частотных гипотез. На вход модуля поступает поток комплексных отсчётов, сформированных после цифровой обработки в приёмном тракте.

Результатом функционирования обнаружителя является поток отсчётов корреляционной функции, формируемый для каждого рассматриваемого частотного сдвига. Данный поток передаётся в существующий тракт прямого доступа к памяти (DMA) и далее записывается в оперативную память системы.

Включение обнаружителя в цифровой тракт приводит к изменению состава передаваемых данных между логической частью (PL) и процессорной системой (PS): вместо исходных отсчётов АЦП в память передаются результаты корреляционной обработки. При этом механизм конфигурирования приёмного тракта и структура интерфейса DMA сохраняются без изменений.

4. Архитектура ПЛИС-обнаружителя

4.1. Общая организация вычислительного тракта

Архитектура обнаружителя разработана в среде Vivado с использованием инструмента проектирования *IP Integrator*, предназначенного для построения системы из готовых IP-ядер и пользовательских модулей с графическим описанием их взаимосвязей.

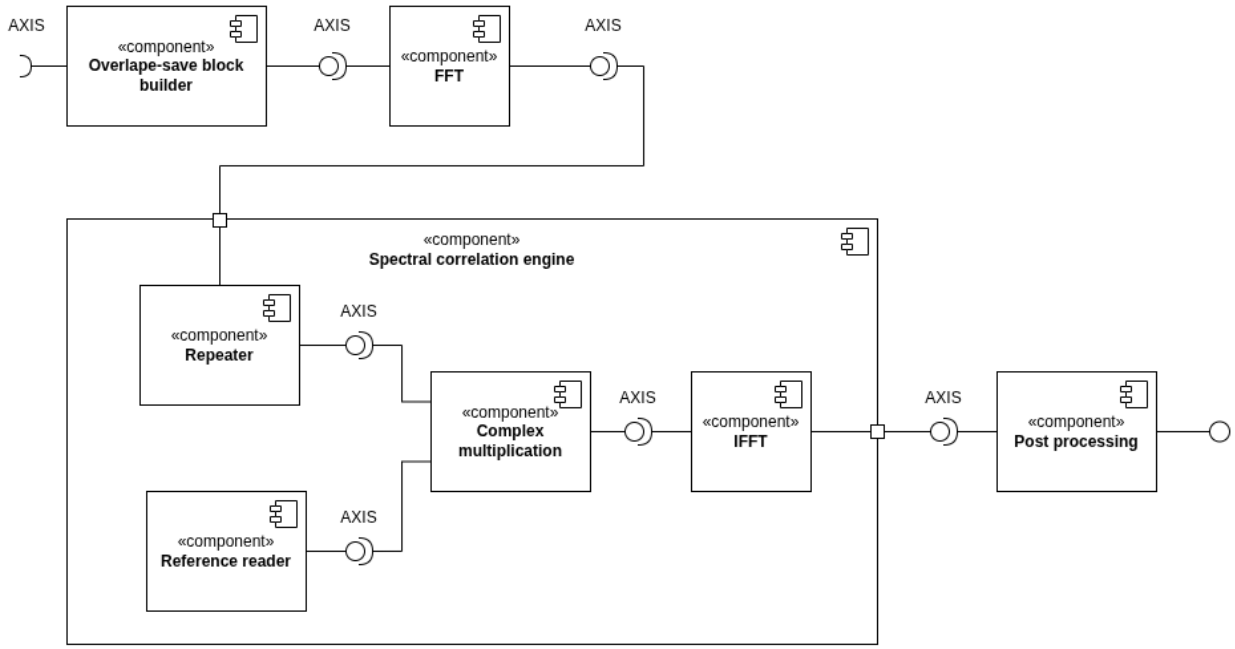


Рис. 2: Диграмма компонентов ПЛИС-обнаружителя.

В основе решения лежит потоковый принцип обработки данных с применением интерфейса AXI4-Stream (AXIS). Передача информации между функциональными блоками осуществляется в виде последовательности отсчётов, объединённых в пакеты фиксированной длины. Обмен сопровождается стандартными служебными сигналами интерфейса AXIS, в том числе сигналом наличия корректных данных (`tvalid`) и сигналом завершения пакета (`tlast`). Такой подход обеспечивает упорядоченную и синхронизированную передачу данных между модулями без использования общей адресной памяти.

Назначение разработанной архитектуры — вычисление взаимной корреляции входного сигнала с преамбулой длиной 480 отсчётов для четы-

рёх различных частотных гипотез. Корреляция вычисляется непрерывно без разрывов между соседними блоками обработки.

Входной поток формируется после децимации сигнала с коэффициентом 26. При тактовой частоте АЦП 10,4 МГц частота дискретизации после децимации составляет 100 кГц. Далее поток комплексных отсчётов поступает в тракт корреляционной обработки.

В работе обнаружителя можно выделить следующие этапы:

1. формирование блока данных для БПФ с учётом перекрытия;
2. прямое БПФ входного сигнала;
3. умножение спектра сигнала на спектры частотно-смещённых преамбул;
4. многоканальное обратное БПФ;
5. удаление некорректной части результата;
6. передача результатов на выходной поток.

4.2. Корреляция через БПФ

Взаимная корреляция дискретных комплексных сигналов определяется выражением

$$R_{xy}[n] = \sum_{m=0}^{K-1} x[m+n]\, y^*[m], \tag{3}$$

где $K = 480$ — длина преамбулы, а y^* обозначает комплексное сопряжение.

Корреляция эквивалентна свёртке сигнала $x[n]$ с временно инвертированным и сопряжённым сигналом $y[n]$:

$$R_{xy}[n] = x[n] * h[n], \qquad h[n] = y^*[-n]. \tag{4}$$

Согласно теореме о свёртке [15]:

$$\mathcal{F}\{x[n] * h[n]\} = X[k] \cdot H[k], \tag{5}$$

где $\mathcal{F}$ — дискретное преобразование Фурье.

Следовательно, корреляцию можно вычислять в частотной области:

$$R_{xy}[n] = \mathcal{F}^{-1}\{X[k] \cdot H[k]\}. \tag{6}$$

При длине блока $N = 2048$ вычислительная сложность прямого расчёта корреляции порядка $O(NK)$ становится существенно выше, чем сложность алгоритма БПФ $O(N \log N)$. Поэтому в архитектуре применяется метод быстрой корреляции через прямое и обратное БПФ.

4.3. Переход от циклической свёртки к линейной

Преобразование Фурье реализует циклическую свёртку. Для получения линейной корреляции используется блочный метод с перекрытием — схема overlap-save, подробно описанная в [15], [17].

Пусть:

- $N = 2048$ — длина БПФ;
- $K = 480$ — длина преамбулы;
- $L = N - K + 1 = 1569$ — число корректных отсчётов на блок.

Каждый входной блок формируется следующим образом:

- первые $K - 1 = 479$ отсчётов — последние отсчёты предыдущего блока;
- оставшиеся L отсчётов — новые данные входного потока.

После выполнения обратного БПФ первые $K-1$ отсчётов результата отбрасываются как искажённые циклической свёрткой. Оставшиеся L отсчётов являются корректным фрагментом линейной корреляции.

В архитектуре это реализуется аппаратно следующим образом:

- модуль **last_samples_buffer** хранит 479 последних отсчётов предыдущего блока;
- модуль **double_buffering** обеспечивает непрерывную подачу 2048 отсчётов на вход БПФ;
- модуль **remove_tail** отбрасывает первые 479 отсчётов результата обратного БПФ.

Таким образом, поток данных организован как последовательность перекрывающихся блоков длиной 2048 отсчётов, что гарантирует отсутствие разрывов и артефактных значений корреляционной функции на границах блоков данных.

4.4. Формирование и хранение спектров преамбул

Для компенсации возможного частотного рассогласования предусмотрена обработка четырёх частотных гипотез. Каждая гипотеза соответствует частотному сдвигу Δf_i во временной области:

$$s_i[n] = s[n]e^{j2\pi\Delta f_i n T_s}. \tag{7}$$

Спектры всех четырёх вариантов преамбулы:

- предварительно вычисляются вне ПЛИС;
- комплексно сопрягаются;
- сохраняются в блочной памяти (Block Memory Generator).

Во время работы системы модуль **reference_reader** синхронно считывает спектральные коэффициенты из BRAM и подаёт их на вход комплексных умножителей.

4.5. Обработка спектра входного сигнала

После формирования блока длиной 2048 отсчётов данные поступают в ядро FFT (Radix-2, Burst I/O). Выходной спектр представляет собой поток комплексных значений, передаваемых через интерфейс AXIS.

Для обеспечения параллельной обработки четырёх частотных гипотез каждый спектральный отсчёт входного сигнала дублируется модулем **repeater**. Таким образом формируется четыре параллельных потока данных, которые далее перемножаются с соответствующими спектрами преамбул в блоке Complex Multiplier.

Поскольку ядро обратного БПФ сконфигурировано как четырёхканальное, далее выполняется упаковка четырёх 32-битных комплексных отсчётов в одно 128-битное слово с помощью AXI4-Stream Data Width Converter. Это позволяет синхронно выполнить четыре независимых IFFT в одном IP-ядре.

4.6. Обратное БПФ и формирование выходного потока

Многоканальное IFFT (размер 2048) формирует четыре временных последовательности корреляции.

Каждый входной пакет длиной 2048 отсчётов преобразуется в 2048 отсчётов результата, из которых 479 являются некорректными согласно методу overlap-save. Модуль **remove_tail** удаляет эту часть, после чего оставшиеся 1569 отсчётов передаются в выходной поток.

Передача организована пакетами AXIS с корректным формированием сигнала TLAST, что обеспечивает согласованность с последующими блоками обработки или средствами моделирования.

4.7. Организация управления потоками

Поскольку ядра FFT используют режим Burst I/O, обработка выполняется блоками фиксированной длины. Для исключения пауз между блоками применена схема двойной буферизации. Пока один буфер передаётся в FFT, второй заполняется новыми входными данными.

Модули:

- **axis_packet_demux**,

- **axis_mux**,
- **buf_controller**,
- **slow_fifo_reader**

обеспечивают согласование скоростей потоков, управление пакетами и корректное формирование управляющих сигналов AXIS.

Таким образом, движение данных в системе описывается следующей схемой:

входной поток → формирование блока → FFT → спектральное умножение → IFFT → удаление хвоста → выходной поток.

5. Тестирование

Проверка корректности функционирования ПЛИС-обнаружителя выполнялась в среде Vivado с использованием режима behavioral simulation. Данный режим моделирования позволяет проверить логическую корректность реализации на уровне RTL-описания без учёта временных задержек, вносимых после этапов синтеза и трассировки.

5.1. Параметры тестовой симуляции

Для проведения моделирования был сформирован тестовый комплексный сигнал с частотой дискретизации

$$f_s = 100 \text{ кГц},$$

соответствующей режиму работы системы после децимации. Сигнал моделировал поток отсчётов АЦП и содержал три копии эталонной преамбулы, распределённые во времени.

В сигнал была внесена искусственная частотная ошибка

$$f_{\text{err}} = -2800 \text{ Гц}.$$

Размер БПФ, используемый в архитектуре обнаружителя,

$$N_{\text{FFT}} = 2048,$$

что определяет частотное разрешение

$$\Delta f = \frac{f_s}{N_{\text{FFT}}} \approx 48.828125 \text{ Гц}.$$

Частотные гипотезы формировались на основе циклического сдвига спектра преамбулы. Первый частотный сдвиг выбирался как ближайший к истинной частотной ошибке. Три последующих сдвига выбирались последовательно дальше от истинного значения ошибки с целью наблюдения деградации корреляционного пика при увеличении остаточной частотной рассинхронизации.

Таким образом обеспечивалась возможность экспериментально продемонстрировать:

- уменьшение амплитуды корреляционного пика при росте остаточной частотной ошибки;
- исчезновение отчётливо выраженного максимума при превышении допустимого уровня рассогласования.

Значения использованных частотных сдвигов составили:

$$[-2783.203125,\ -2734.375,\ -2685.546875,\ -2636.71875]\ \text{Гц}.$$

Основные параметры тестовой симуляции приведены в таблице 1.

Параметр	Значение
Частота дискретизации f_s	100 кГц
Размер БПФ N_{FFT}	2048
Частотное разрешение Δf	48.828125 Гц
Внесённая частотная ошибка f_{err}	-2800 Гц
Количество преамбул в тестовом сигнале	3
Количество частотных гипотез	4
Значения частотных сдвигов	-2783.203125 Гц -2734.375 Гц -2685.546875 Гц -2636.71875 Гц

Таблица 1: Параметры тестовой RTL-симуляции

5.2. Организация моделирования

Спектр эталонной преамбулы был предварительно вычислен в среде Python. На его основе были сформированы файлы инициализации памяти формата `.coe`, использованные для загрузки данных в BRAM модуля `reference_reader`.

Аналогичным образом входной тестовый сигнал, имитирующий поток отсчётов АЦП, был подключён к входу обнаружителя в составе тестового окружения.

В ходе behavioral simulation:

- выполнялось формирование перекрывающихся блоков длиной 2048 отсчётов;
- осуществлялось прямое БПФ, спектральное умножение и обратное БПФ;
- результаты расчёта взаимнокорреляционной функции сохранялись во внешний файл.

Сохранённые данные обрабатывались Python-скриптом, который выполнял визуализацию корреляционных функций для всех частотных гипотез и сравнение с результатами, полученными полностью в программной модели.

5.3. Результаты

Для первой частотной гипотезы, выбранной как ближайшая к истинной частотной ошибке, наблюдались три отчётливых корреляционных пика, соответствующих числу преамбул, встроенных в тестовый сигнал.

Для второй гипотезы, характеризующейся несколько большей остаточной частотной ошибкой, также сохранялись три различимых корреляционных пика, однако их амплитуда была заметно ниже по сравнению с первой гипотезой. Это отражает частичную потерю когерентности при увеличении частотного рассогласования.

Для третьей и четвёртой частотных гипотез выраженные корреляционные максимумы уже не наблюдались. Выходной сигнал имел шумоподобный характер без отчётливо выделяющихся пиков, что свидетельствует о превышении допустимого уровня остаточной частотной ошибки, при котором энергия корреляционного отклика рассеивается и не формирует локального максимума.

Полученные результаты полностью совпали с расчётами, выполненными в программной модели, что подтверждает функциональную корректность реализации алгоритма корреляции и механизма обработки частотных гипотез на уровне RTL-описания.

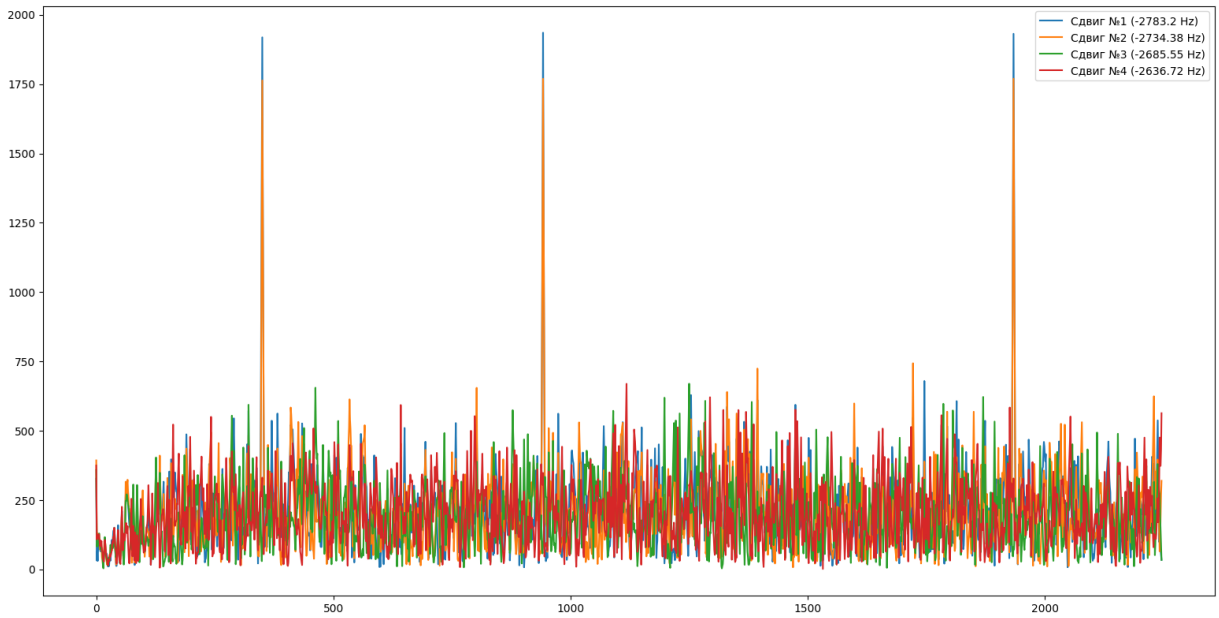


Рис. 3: Корреляционные функции для четырёх частотных гипотез

Заключение

В ходе выполнения выпускной квалификационной работы получены следующие результаты.

1. Выполнен обзор существующих методов обнаружения цифровых сигналов и особенностей их реализации на программируемых логических интегральных схемах. Установлено, что корреляционный метод является теоретически обоснованным и практически применимым для обнаружения сигналов с известной структурой преамбулы. Для аппаратной реализации выбран подход, основанный на вычислении взаимнокорреляционной функции с использованием быстрого преобразования Фурье в соответствии с теоремой о свёртке, что обеспечивает эффективность реализации.

2. Спроектирована архитектура ПЛИС-подсистемы обнаружения, предназначенной для интеграции в устройство Pluto SDR+. Разработанная архитектура реализует вычисление взаимнокорреляционной функции входного сигнала с эталонной преамбулой для набора частотных гипотез и учитывает особенности существующего цифрового тракта приёма и передачи данных между логической

частью (PL) и процессорной системой (PS).

3. Выполнена реализация обнаружителя на языке VHDL для ПЛИС, входящей в состав SoC семейства Xilinx Zynq-7000. Разработанный модуль интегрирован в штатную HDL-прошивку устройства Pluto SDR+. Реализация выполнена с использованием потоковой обработки данных и интерфейса AXI4-Stream, обеспечивающего передачу результатов корреляционной обработки в тракт DMA.

4. Проведено функциональное тестирование разработанной подсистемы средствами RTL-моделирования в среде Vivado. По результатам симуляции подтверждена корректность вычисления взаимнокорррреляционной функции и работоспособность реализованного решения. Полученные данные сопоставлены с результатами программного моделирования, что показало их согласованность.

Список литературы

[1] ADALM-PLUTO Evaluation Board [Electronic resource]. — URL: https://www.analog.com/en/resources/evaluation-hardware-and-software/evaluation-boards-kits/adalm-pluto.html (дата обращения: 1 ноября 2025 г.).

[2] AMD Vitis IDE. — URL: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis.html (дата обращения: 6 января 2025 г.).

[3] AMD Vivado IDE. — URL: https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vivado.html (дата обращения: 6 января 2025 г.).

[4] AMD Zynq 7000. — URL: https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-7000.html (дата обращения: 6 января 2025 г.).

[5] Adams, L. Choosing the Right Architecture for Real-Time Signal Processing Designs. White paper. Texas Instruments. 2002 [Electronic resource]. — URL: https://www.ti.com/cn/lit/wp/spra879/spra879.pdf (дата обращения: 1 ноября 2025 г.).

[6] Altera Cyclone IV FPGA Device Family Overview [Electronic resource]. — URL: https://cdrdv2-public.intel.com/654630/cyiv-51001.pdf (дата обращения: 1 ноября 2025 г.).

[7] Collection of open-source VHDL/Verilog IP cores. — https://github.com/fabriziotappero/ip-cores. — 2026. — Дата обращения: 2026.

[8] FPGAs for High-Performance DSP Applications [Electronic resource]. — URL: https://cdrdv2-public.intel.com/650321/wp_dsp_comp.pdf (дата обращения: 1 ноября 2025 г.).

[9] FuseSoC Core Directory. — https://cores.fusesoc.net/. — 2026. — Дата обращения: 2026.

[10] General-Purpose TigerSHARC Processor Product Brief [Electronic resource]. — URL: http://analog.com/media/en/news-marketing-collateral/product-highlight/4667563674187gptigersharc(b)final.pdf (дата обращения: 1 ноября 2025 г.).

[11] IEEE Standard for SystemVerilog [Electronic resource]. — URL: https://ieeexplore.ieee.org/document/10458102 (дата обращения: 1 ноября 2025 г.).

[12] Linaro Linux. — URL: https://old.linaro.org/core-technologies/linux-kernel/ (дата обращения: 6 января 2025 г.).

[13] Mengali Umberto. Synchronization Techniques for Digital Receivers. — Springer, 1997.

[14] OpenCores: Open Source IP Cores. — https://opencores.org/. — 2026. — Дата обращения: 2026.

[15] Oppenheim Alan V., Schafer Ronald W. Discrete-Time Signal Processing. — 3 edition. — Upper Saddle River, NJ : Prentice Hall, 2010.

[16] RF transceiver AD9363. — URL: https://www.analog.com/en/products/ad9363.html (дата обращения: 6 января 2025 г.).

[17] Smith Steven W. The Scientist and Engineer's Guide to Digital Signal Processing. — San Diego, CA : California Technical Publishing, 1997.

[18] Software-Defined Radio Solutions from Analog Devices [Electronic resource]. — URL: https://www.analog.com/media/en/news-marketing-collateral/solutions-bulletins-brochures/software-defined-radio-solutions-from-adi.pdf (дата обращения: 1 ноября 2025 г.).

[19] TMS320C64x Technical Overview [Electronic resource]. — URL: https://www.ti.com/lit/pdf/spru395 (дата обращения: 1 ноября 2025 г.).

[20] VHDL 2019 standard. — URL: https://ieeexplore.ieee.org/document/8938196 (дата обращения: 6 января 2025 г.).

[21] Verilog 2005 standard. — URL: https://ieeexplore.ieee.org/document/1620780 (дата обращения: 6 января 2025 г.).

[22] Xilinx 7 Series FPGAs Data Sheet: Overview [Electronic resource]. — URL: https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview (дата обращения: 1 ноября 2025 г.).

[23] Xilinx Linux Kernel. — URL: https://github.com/Xilinx/linux-xlnx (дата обращения: 6 января 2025 г.).

[24] Zynq-7000 SoC Data Sheet: Overview [Electronic resource]. — URL: https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview (дата обращения: 1 ноября 2025 г.).

[25] Гришин Ю. П., Ипатов В. П., Казаринов Ю. М. Радиотехнические системы / Ed. by Ю. М. Казаринов. — Москва : Высшая школа, 1990.